

Práctica Análisis Léxico

Lenguajes de programación y procesadores de lenguaje

24 de Septiembre de 2026

En el Campus Virtual de la asignatura en el apartado de "Laboratorio 1: Análisis Léxico" encontraréis un enlace para obtener JFlex como archivo ".jar" y otro para obtener el código Java para generar el analizador léxico en "alex.zip". También encontraréis un ejemplo de especificación del léxico usando JLex "mini_lexico.l" y varios ejemplos de entrada para el procesamiento final en "input.txt", "input_1.txt" y "input_erroneo.txt". Descargad todos estos archivos en una carpeta y descomprimid "alex.zip".

Para más información o versiones más recientes de JLex visitad <https://www.jflex.de/>.

Podéis instalar JLex en vuestra cuenta usando el archivo ".jar" o lo podéis usar directamente en la línea de comandos. También podéis utilizar el entorno para desarrollo de código Java que preferáis. Deberéis definir los caminos necesarios en el CLASSPATH o usar cp al llamar a java.

Generación de un analizador léxico sencillo

En esta práctica vamos a desarrollar un analizador léxico para un lenguaje muy sencillo que incluye únicamente:

- Tipos de variables enteras (**ent**) y booleanas (**bool**). Los posibles valores de los booleanos son **verdadero** y **falso**.
- Asignación de variables a través del operador **=**.
- Operador suma **+**, igualdad (**==**) y mayor (**>**).
- Condicionales: de la forma

```
si (Expr) [  
  LIns  
] sino [  
  LIns  
]
```

- Comentarios de la forma **#...#**.

Un ejemplo de programa sencillo en este lenguaje sería el siguiente (incluido en el fichero **input.txt**):

```
# Este es un ejemplo #  
ent x;  
ent y;  
bool b;  
x = 13;  
y = 2;  
si (x == y) [  
  b = falso;  
] sino [  
]  
b = x == y + 2;
```

En este laboratorio vamos a ver cómo podemos utilizar JFlex para generar automáticamente un analizador léxico para este lenguaje. Para ello en primer lugar vamos a observar el fichero `mini_lexico.l`. Este fichero contiene la especificación del lenguaje.

1. Utilizando JFlex generamos automáticamente un fichero JAVA incluyendo el analizador léxico generado. Para procesar el archivo `mini_lexico.l` con JFlex, tenemos que ejecutar:

En SO LINUX: `$ java -cp jflex.jar jflex.Main mini_lexico.l`

Esto generará un archivo `AnalizadorLexicoTiny.java` que contendrá el analizador. Vamos a echar un ojo a la salida de `jflex` y el fichero que generamos, podemos ver un poco de información sobre el autómata que estamos generando, como su número de estados o posibles warnings.

2. Debemos mover el analizador léxico `AnalizadorLexicoTiny.java` dentro de la carpeta `alex`. En ella se encuentran otros elementos necesarios para desarrollar el analizador como los ficheros `AlexOperations.java` y `ClaseLexica.java`. En ellos se definen las operaciones para crear las distintas unidades léxicas.

Compilamos el código java de la carpeta "alex":

En SO Linux: `$ javac alex/*.java`

y ya podemos usar el analizador.

3. En la versión del Main que se adjunta, se lee de la entrada de un fichero que se pasa por parámetro. El fichero `input.txt` contiene el ejemplo mencionado anteriormente.

Para analizarlo debemos ejecutar:

En SO Linux: `$ java alex.Main input.txt`

desde el directorio de trabajo y obtendréis los tokens de la entrada (puede ser necesario añadir `cp` si no tenéis "." en el `CLASSPATH`). Podemos comprobar que los tokens y la unidad léxica a la que pertenecen son los esperados.

También podemos probar a modificar la entrada e introducir errores léxicos, el analizador será capaz de reconocerlos.

Posibles cambios a realizar

Ahora vamos a realizar algunos cambios en nuestro lenguaje extendiéndolo. De cara a realizarlos deberemos:

1. Modificar el fichero `.l` añadiendo las nuevas expresiones regulares
2. Modificar el fichero `ClaseLexica.java` añadiendo las nuevas unidades léxicas
3. Modificar el fichero `AlexOperations.java` añadiendo los constructores para las nuevas unidades léxicas

Algunos cambios que podéis realizar (ordenados de más sencillos a más complejos) son los siguientes:

- Añadir el operador para la multiplicación
- Añadir otros operadores relacionales y las operaciones entre booleanos

- Añadir instrucciones para permitir bucles
- Permitir el símbolo `_` en los identificadores pero no al principio, ni final ni dos seguidos
- Añadir números reales y el tipo de número real

Recuerda que de cara al proyecto final será necesario extender el lenguaje para que incluya las instrucciones básicas presentes en la mayoría de los lenguajes de programación, así como las incluidas en los requisitos vía carta.